

Batcher: Single-Node Speed That Scales

One Mergeable Algebra from a Core to a Cluster

Stephen Offer

Anyscale • stephen.offer@anyscale.com

Abstract

The data engines most teams run were designed for a world that no longer exists. Spark’s architecture assumes a machine too small to hold the problem, so it distributes everything and charges for it on every query. DuckDB and Polars assume the opposite and are wonderfully fast until the data outgrows the box. Both optimize once, before the query runs, from statistics that were stale before it was written. And all of them treat machine learning as somebody else’s job: a second system, a second copy of the data, and a serialization boundary between.

Batcher rests on three bets. **One algebra, every scale:** every stateful operator exists once, as a mergeable triple partial $\rightarrow$ combine $\rightarrow$ finalize in Rust over Apache Arrow, and one core, ninety-six cores and a cluster differ only in how it is scheduled. Small queries pay nothing for distribution; large ones need no rewrite; and batch becomes the bounded case of streaming rather than a second execution model. **An engine that learns:** Batcher measures what every operator did and plans the next query from that evidence instead of from vendor constants, with every learned value provably unable to change an answer. **One engine for the whole pipeline:** relational queries, image and audio decode, embeddings, vector search and GPU inference are expressions over the same Arrow batches, under the same optimizer.

On one 96-core machine Batcher matches DuckDB on TPC-H [33] sf1, leads Polars and Daft by 1.8–2.2 $\times$, sweeps PyArrow’s Acero engine, and wins 43 of 43 ClickBench queries against DuckDB on identical input. On a cluster, aggregate shuffle throughput rises superlinearly: 7.6 $\times$ for 4 $\times$ the nodes. We are equally clear about the ceilings, because a roadmap is more useful than a scoreboard.

1 The assumptions that expired

Every architecture encodes the world it was born into. The distributed data stack most companies run was designed between 2009 and 2014. It rests on four assumptions. All four were reasonable then. None of them holds now.

“The data will not fit on one machine.” True in 2012, when a large server had 16 cores and 64 GB. Everything followed from it: a cluster you start before you ask a question, a scheduler between you and your data, serialization on every hop. A single cloud instance now has 96 to 192 cores and up to twelve terabytes of memory. The machine that needed a rack fits in a footnote, and the tax stayed anyway, which is why a query that

would take twelve milliseconds in-process still takes seconds through a distributed runtime.

“We know what the data looks like.” Classical optimizers were built for curated schemas, statistics refreshed on a schedule, and queries written once and run for years. That world had ANALYZE jobs and DBAs. This one has schema drift, object storage, and JSON columns, and the same query never sees the same distribution twice. Optimizing once from a stale catalog stopped being a performance choice. It is a guess.

“Analytics and machine learning are separate systems.” They were, for good reasons: different tools, different hardware, different teams. So the standard pipeline is a warehouse query that writes Parquet, a framework that reads it back, a GPU cluster that reads it again. Most of the wall-clock time in a modern AI pipeline is not computation. It is copying.

“Structured data is the data.” This is the one the others rest on. IDC put the global datasphere at 33 zettabytes in 2018 and forecast 175 by 2025 [27], and most of that growth is not rows and columns. It is documents, logs, images, audio, video, sensor traces, and the embeddings derived from them. Analyst estimates commonly place the unstructured share of enterprise data at 80 to 90 percent, and IDC forecasts the split explicitly [13]; we quote it as an estimate rather than a measurement, because it is a projection and not a census.

The share matters less than the change in role. Unstructured data stopped being an archive you queried occasionally. It became the input to the models the business runs on, which puts it on the critical path of pipelines that used to be purely relational. Four boundaries blurred as a result, and each of them used to be an architectural seam: batch and streaming, structured and unstructured, data engineering and machine learning, query and model. Spark saw the first one and answered it well. An engine designed around any one of them has to treat the other side as foreign.

Those four assumptions are the whole argument. The engines built on them are still the default, and the workarounds have become folklore: keep a cluster warm so small queries are bearable, hand-tune the shuffle partition count, maintain a second copy of the data for the ML team. Table 1 puts the generational shift in one page.

2 Bet one: one algebra, every scale

Ask an engine builder how a GROUP BY works. You get two answers. On one machine it is a hash table. On a cluster it is a shuffle, a partial pass, and a reduce. The two share a name, a

The 2010s assumption	What changed	Batcher’s answer
One machine is too small, so distribute everything and pay the tax always.	A single node now has 96–192 cores and terabytes of RAM. Most queries never needed a cluster; some still do.	One mergeable algebra. Distribution is a per-query scheduling decision, not a runtime. No tax on the small case, no rewrite on the large one.
Optimize once, before execution, from catalog statistics.	Schemas drift, storage is object storage, and the same query never sees the same data twice.	Plan from what was <i>measured</i> : sketches built during execution, cost coefficients refit from runtime, strategy chosen by a bandit over outcomes.
Memory limits are a static configuration the operator tunes by hand.	Workloads are heterogeneous—wide strings, embeddings, images—so one setting cannot fit them.	Learn each operator family’s footprint as a per-row ratio and size admission, spill, and morsels from it.
Analytics and ML are separate systems joined by files.	The workload is one pipeline: read, transform, infer, write.	Decode, embed, search, and infer as expressions over the same Arrow batches, in the same optimized plan.
Each data shape gets a purpose-built engine; each model tier a purpose-built framework.	One pipeline touches Parquet, JSON, raw text and images at once, and one team runs regressions, deep models and LLMs.	One engine across four data shapes and four workload tiers, under one optimizer and one memory manager (Figure 3).

Table 1: Why a new architecture, rather than a faster implementation of the old one. Each row is a design decision the previous generation made correctly for its era, and a consequence of that decision which is now the thing slowing teams down.

SQL semantics, and not a line of code.

That is the root of the bifurcation. Two implementations means two performance profiles and two sets of bugs. Eventually it means two different answers. A float key that hashes one way locally and another after a shuffle splits one group in two, and the single-node test suite stays green because the single-node implementation is correct.

Batcher makes that split impossible. A stateful operator may exist only as a mergeable triple, and the three execution regimes vary only its scheduling (Figure 1). One core folds a stream of batches. Many cores partition and merge. Many machines partition, run the partial pass per node, ship pre-aggregated state, and combine in a tree. Formally, for any partitioning $\{p_k\}$ of an input P ,

$$\text{combine_finalize}(\bigcup_k \text{partial}(p_k)) = \text{finalize}(\text{partial}(P)), \quad (1)$$

which is a property test rather than a comment: a generator produces random partitionings and asserts equality against the single-pass result, and a separate test asserts that a distributed run and a single-node run of the same plan return identical rows.

Two things follow, and they are the paper’s thesis. **There is no distribution tax on the small case**, because distribution is a scheduling decision made per query rather than a runtime you start. **And there is no rewrite on the way out**, because the operator that wins on one core is the operator that runs on eight nodes. Since mappers publish state that is already reduced rather than raw rows, per-node memory stays bounded however wide the cluster grows, so shuffle throughput rises with node count instead of collapsing into contention (Figure 5).

The triple itself is old: Gray et al.’s algebraic aggregate [12] and MapReduce’s combiner [7]. Every distributed engine reaches for it somewhere, usually for aggregation. What is different is treating it as an *admission criterion* for every stateful operator, join build, window and top- N included, so that no second implementation ever exists to drift from the first. The cost is real and we pay it: an algorithm with no mergeable form does not get added, and Section 11 names the operator where that still hurts.

3 Batch is the bounded case

The same constraint settles a second split, and this one is usually presented as a feature rather than an accident: most engines have a batch execution model and a streaming execution model, and reconcile them at the API.

Batcher reconciles them lower down. Its executor is a streaming executor for *every* query. Operators pull small batches through a pipeline and materialize only at breakers, so peak memory is the breakers’ state plus one batch per worker: a constant, independent of how much data arrives. On a join-heavy shape that is 3.4 MB at one million rows and 3.3 MB at four million, where the materializing path grows from 198 MB to 794 MB. The bounded case is not a special mode. A finite table is simply a stream that ends.

That matters more than it sounds, because the mergeable triple *is* the incremental form. An aggregate that folds each batch through partial and combine never reads its whole input, which is exactly what a continuous query needs: state that updates per arrival and finalizes on demand. The operator a batch query uses and the operator a continuous query needs are not two implementations that happen to agree. They are one implementation, and the difference is whether anything ever calls finalize. The same property makes an operator a pipeline breaker for *ordering* without making it a materialization point for *memory*, which is why a streaming aggregation over an unbounded source has a memory bound at all.

The practical payoff is that the memory story does not change with the source. A query over a hundred terabytes of Parquet and the same query over a Kafka topic run the same operators under the same bound, and a pipeline that outgrows its batch window does not get rewritten to survive.

What is genuinely unfinished. The execution model is unified; the *continuous runtime* around it is not, and it would be misleading to let the first imply the second. Batcher’s continuous mode is micro-batch. The watermark is a single global scalar on the driver rather than a minimum over per-partition

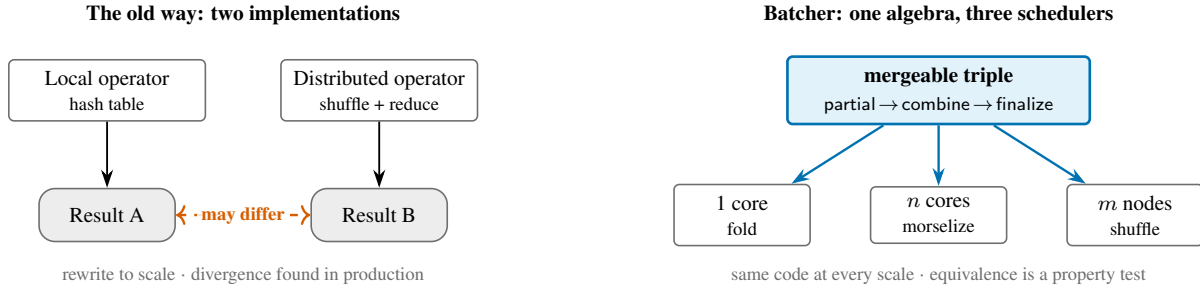


Figure 1: The decision the rest of the system is built on. Batcher admits an operator only if it can be written as a mergeable triple, a partial pass, an associative and commutative combine, and a finalize, and then varies only the *scheduling*. Aggregation, distinct, join build, window, and top-*N* all qualify. There is no second implementation to disagree with the first, so scaling up is a configuration change rather than a migration.

watermarks, so one fast partition can drag it forward and cause a slow partition’s rows to be dropped as late. Operator state is an in-memory Arrow batch checkpointed by rewriting it in full, to local disk. Exactly-once holds for one lakehouse format and not in general. Spark’s Structured Streaming is a decade ahead here and we would not advise replacing it today. What we claim is narrower and, we think, the harder half: the execution model underneath is already the streaming one, so the remaining work is a state backend and a watermark discipline rather than a second engine.

4 Bet two: an engine that learns

The second consequence of refusing to fork is the one that compounds. Because the same operator runs everywhere, **a measurement taken anywhere is valid everywhere**. A cardinality observed on one core is evidence about the same operator on ninety-six, which is what makes it sound for an engine to plan from its own history.

Every other engine in Table 2 ships its cost model as constants, chooses join strategy by heuristic, and sizes memory from a plan estimate. Those constants were tuned once, on hardware nobody here is running, against data nobody here has. Batcher replaces each with a measurement: cardinality from sketches built during execution and the true row counts of previous runs, cost coefficients refit from observed operator times, physical strategy from a bandit over measured outcomes, and memory from each operator family’s real footprint.

One rule makes this safe rather than alarming. **Every learned value is result-invariant**. Each bandit arm emits the same relation, a threshold picks between equivalent algorithms, a partition count only shards. Learning changes how much memory a query reserves, when it spills, how wide a batch is, and which of several equivalent algorithms runs. It can never change the answer. That is property-tested, and a cold store reproduces the untuned engine byte for byte, so the worst a stale value can do is cost throughput. This is a deliberately different bet from learned optimizers that predict plans [17, 21, 22], which change which plan runs in ways that are hard to bound. A weaker signal buys a far stronger safety property, which is what lets the feature ship on by default.

4.1 Three clocks, not one

Adaptive query processing usually means one thing: re-plan at a stage boundary. Batcher does that too. It is the least interesting of the three places it adapts.

Microseconds: the filter reorders itself. A conjunctive filter evaluates its cheapest predicate first, which is what a static cost model can tell you. But cost is not selectivity. The case a static model cannot fix even in principle is two predicates of identical cost where only one is selective. Write the broad one first and nothing gets removed, so the selective one runs at full width anyway.

DuckDB solves this by hill-climbing: swap a random adjacent pair, keep the swap if total runtime improved over the next ten batches. Batcher reaches the answer differently. Because the short-circuit evaluates conjuncts one at a time, it can attribute rows and nanoseconds to each conjunct *individually*, and order by time-per-row over fraction-removed. The implied order arrives after a **single morsel** where a hill-climb needs tens of batches to walk the permutation. On the adversarial shape, two regular-expression matches of identical static cost with the broad one written first, that is 48.64 ms down to 28.18 ms. Correctness is free here: the conjuncts of an AND commute, so every order yields the identical mask. Two workers may briefly disagree about the best order at no cost, which is why the shared state needs no lock.

Microseconds: the aggregate picks its own algorithm. A parallel aggregate normally hashes each morsel into its own table and merges the tables. That is right when grouping reduces. It is wrong when grouping does not: `GROUP BY l_orderkey, l_linenum` over TPC-H `lineitem` reduces nothing at all, so every partial row survives and the merge inherits the whole relation. The alternative is to partition the input by key first and aggregate each partition exactly once. Measured on a 60M-row table across 96 cores, the two paths cross just under 0.18 rows kept per input row:

rows kept per input row	0.10	0.34	1.00
partial → combine (ms)	125.7	350.9	1351.6
partition → aggregate (ms)	198.2	185.3	370.2

Conventional: three systems, two copies



Batcher: one plan, one optimizer, zero copies

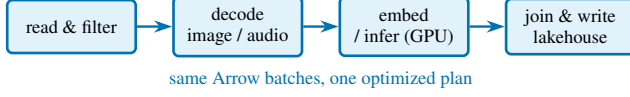


Figure 2: Why the seams cost more than the compute. In the conventional stack each handoff is a full serialize, a write, a read and a deserialize of the whole dataset. No optimizer sees across the boundary. In Batcher the decode, the embedding, and the inference are expressions in the same plan as the filter and the join, so a predicate can be pushed below a decode and a tensor never leaves the engine.

Two things make this more than a tuned threshold. First, the choice comes from a *measurement* and not an estimate. The optimizer’s distinct-count for a group key is a sketch, and after a filter it is a guess about a distribution nobody has looked at. So the executor partials a sample of morsels and reads the reduction those partials actually achieved. If grouping does reduce after all, the sample is handed straight back to the reducing path. No work is wasted either way.

Second, look at what the alternative path is. `partition → partial → finalize` is the exact composition Batcher runs *across machines*, executed here across cores. The high-cardinality single-node aggregate is the distributed algorithm, chosen at runtime. This is the constraint paying for itself. Refusing to fork the operator did not just prevent a class of bug. It handed the single-node executor a second strategy for free.

Stages and runs. The other two clocks are the familiar ones. At a pipeline breaker the engine has measured what it just processed, so it re-plans the remainder on real cardinalities. Across executions it accumulates sketches, refits cost coefficients and converges bandits. The comparison is the point: DuckDB adapts on the first clock, Spark AQE on the second, and neither has the third. Adaptivity is usually discussed as one feature; it is three, and they correct different errors. A static cost order that guessed wrong about selectivity. An optimizer estimate that guessed wrong about a join’s size. A shipped constant that was never right for this hardware.

5 Bet three: one engine for the whole pipeline

The third bet is the one users see first, and it has two axes.

Engines specialize by *data shape*. DuckDB and Polars are superb over structured columns and have no image or audio surface at all. Ray Data and Daft handle multimodal data well and have no relational optimizer. Spark covers structured and semistructured, and its multimodal story is a UDF. A team whose pipeline touches Parquet, JSON, raw text and JPEGs in one job,

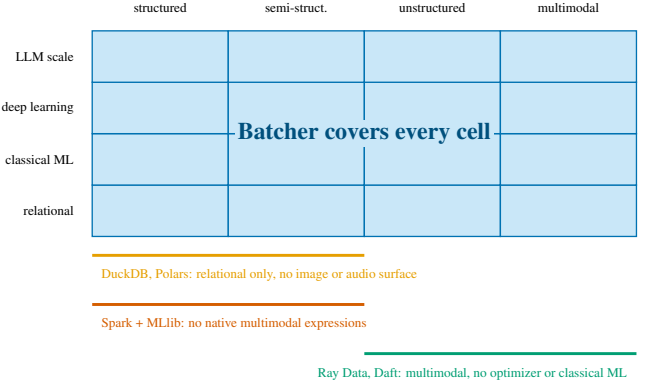


Figure 3: The second fragmentation. Engines specialize by *data shape* across the top and by *workload tier* down the side; the coloured spans show how far each comparator reaches. No system we surveyed covers both axes natively inside one optimized plan. Batcher fills the grid: 30 file and table formats across the four shapes, and 125 public ML entry points from generalized linear models and mixtures through deep-model inference to LLM batch generation.

which is most teams, therefore runs several systems and writes the glue.

They specialize by *workload tier* too: classical machine learning in one framework, deep-model inference in another, LLM generation in a third, and none of them is the system holding the data. Batcher covers 30 file and table formats across the four shapes and 125 public ML entry points, from generalized linear models and mixtures through GPU inference to LLM batch generation with request packing and structured output. Figure 3 is the picture, and the claim it supports is bounded: not that no engine can reach any given cell, but that no engine we surveyed reaches all of them *natively, in one plan, under one optimizer*.

In Batcher, preparing data for a model is not a different kind of work from querying it. Image decode, crop and normalization to a tensor are Arrow kernels in the data plane, not a Python UDF. So are audio decode, resampling, mel-spectrogram and MFCC, the last two matched to `torchaudio` within 10^{-6} and, among the general-purpose engines we surveyed, without an in-engine equivalent anywhere else. So are vector distances, embedding pooling and fuzzy string matching. Vector search is a plain query. Batch inference runs against pools that load a model once per session and reuse it, with CPU decode and GPU compute overlapped so the accelerator is not idle while a JPEG is unpacked.

One example of what living inside the engine buys: shuffling a training corpus normally means materializing a permuted index of every sample, hundreds of gigabytes at ten billion samples, and it must exist before the first batch is read. Batcher replaces the list with a keyed bijection, a small Feistel network [20] with cycle walking [4], so the epoch order is computed rather than stored. Driver memory is constant in corpus size and a mid-epoch resume is arithmetic, the same idea as [25] with world-size-independent ordering.

The consequence is Figure 2. A retrieval pipeline, read a corpus, strip HTML, chunk, embed, write to a lakehouse table, is one plan and one pass over the data. So is a multimodal one: read images from object storage, decode and normalize, run a

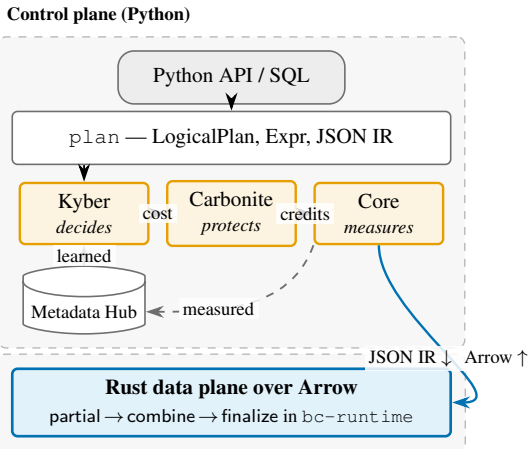


Figure 4: The contract loop. Python builds and optimizes a plan and ships it as JSON IR; all per-row work happens in Rust over Arrow and returns zero-copy. *Core measures*, *Kyber decides*, *Carbonite protects*, and the three cannot import one another, enforced on every commit. That lint is what stops the optimizer from quietly collecting its own telemetry, which compiles and passes tests while corrupting the loop.

model, join the predictions against a customer table, then `MERGE` into Delta. The filter that removes 90% of the rows runs *before* the decode, because one optimizer sees both. Others can do each of these things, and Daft in particular has a strong multimodal surface; what is unusual is that they are all expressions in one optimized plan over one columnar format, governed by one optimizer and one memory manager.

6 Architecture

Batcher is a Python control plane over a Rust data plane, with Arrow `RecordBatch` the only columnar contract at every internal boundary and across the FFI: 97,389 lines of Rust across 14 crates, 228,164 of Python. Python builds a plan, optimizes it, and ships it as JSON; data returns zero-copy through the Arrow C Data Interface. Python never iterates a row, which lets the control plane be a high-level language while nothing it does sits on a per-row path.

The three subsystems in Figure 4 are isolated by a lint contract that forbids them importing one another. The rule reads as bureaucratic until you see what it prevents: an optimizer pass that starts collecting its own telemetry, or an executor path that starts making optimization decisions, will compile and pass every test while silently corrupting the feedback loop the design depends on.

Two execution tiers share one expression type: an interpreter that is the correctness oracle, and a compiled fast path that must be bit-for-bit identical on the subset it supports and fall back silently otherwise. Mid-development one commit made the interpreter raise on integer overflow while the compiler still wrapped, so the same expression returned a wrapped negative or an error depending on whether it happened to compile. Nothing about that is visible in a result. The test that runs both tiers on one input went red immediately.

7 How Batcher compares

Three systems set the bar Batcher is measured against: DuckDB single-node, Daft on Arrow, and Spark distributed. Two more bracket the space. **DuckDB** [26] is the single-node bar: vectorized, a compressed native store with zone maps, optimized once before execution, and it does not distribute. **Spark** [1, 35] distributes, and its Adaptive Query Execution [29] re-plans on measured statistics at stage boundaries; its unit of movement is a row-shuffle over JVM serialization, and its fixed per-query cost is large. **Ray Data** [32], on Ray [24], sits in a different category: a streaming runtime for ML pipelines, with actor pools for GPU work and no relational optimizer. We benchmark against it throughout, because a data engine that claims to serve AI workloads should be held to the standard of the runtime those workloads use today. It is a yardstick here, not a rival. **Daft** [9] and **Polars** [34] are native Rust engines over Arrow [30]; Daft distributes over Ray, Polars does not.

Table 2 places Batcher against all of them, and the pattern in it is the paper’s thesis in tabular form. Batcher does not lead because its kernels are faster than everyone else’s, on several rows it is at parity, and on storage it is behind. It leads on the rows where the previous generation shipped a constant and Batcher ships a measurement, and on the rows where the previous generation required a second system.

8 What changed since Spark

Spark’s authors got the hard parts right, and several of its ideas are load-bearing here. Treating a streaming query as a batch query over an unbounded table is the correct framing and we adopted it. Adaptive Query Execution was the first widely deployed argument that a plan should change once real cardinalities are known. Whole-stage code generation established that an analytical engine should compile rather than interpret its inner loops. None of what follows says those were mistakes. They were right for the hardware and workloads of the time, and three things about both have changed.

The unit of movement. Spark moves rows through JVM serialization, which was reasonable when no ecosystem agreed on a columnar format. Arrow settled that. Batcher moves columnar batches with no serialization step: the memory one operator wrote is the memory the next one reads, and the boundary to Python is a pointer handoff. This is the largest single source of the gap in Table 3, and it is an ecosystem change more than an engineering one.

The cost of being distributed. Spark’s fixed per-query overhead buys fault tolerance and elasticity that matter above a certain size. Below it, the overhead is pure cost, and the threshold moved: the workload that needed a rack in 2012 fits one instance. Batcher treats distribution as a per-query scheduling decision over one algebra. Spark cannot make that choice, because its local and distributed paths are the same path.

Where the statistics come from. Catalyst optimizes from catalog statistics and AQE corrects at stage boundaries. Both assume the catalog is a reasonable prior; on object storage with drifting schemas it often is not, and the correction arrives only

	Batcher	DuckDB	Polars	Spark	Ray Data	Daft
Single-node / distributed	one algebra	single-node	single-node	two paths	two paths	two paths
Optimizer	375 rules, bushy DP	rules + DP	rules	Catalyst	n/a	rules
Cardinality from	sketches + measured	table stats	estimates	table stats	n/a	estimates
Cost model	refit from runtime	static	static	static	n/a	static
Physical strategy	bandit over past runs	heuristic	heuristic	heuristic	n/a	heuristic
Memory sizing	learned per-row	static	static	static	static	static
Re-optimizes mid-query	stage boundary	no	no	stage boundary (AQE)	n/a	no
In-engine audio / mel	native kernels	none	none	none	UDF	UDF
Vector search in SQL	yes	no	no	no	n/a	no
GPU batch inference	warm pools	none	none	external	actor pools	actor pools
Native store	none (Arrow only)	compressed	none	Parquet	none	none
String representation	no StringView	StringView	StringView	UTF8	—	—

Table 2: Batcher’s competitive set is DuckDB, Daft and Spark; Polars and Ray Data are included as reference points, the latter an adjacent streaming ML runtime rather than a relational engine, so its relational rows read n/a rather than as gaps. Blue marks a capability no other column has; amber marks a gap. The distinctive block is the middle: everything derived from measured runs rather than shipped as a constant, and a native AI surface inside the same optimized plan. The distinctive *deficit* is storage (no compressed native format and no string view) and Section 12 shows it landing exactly where this table predicts.

at a shuffle. Batcher keeps that correction and adds a clock on either side of it (Section 4.1). The one Spark has no equivalent for is the outer clock: a query that has run before should not re-derive its plan from the same cold priors.

Where Spark is still ahead, and it is not close. Its fault tolerance is a different class of thing: shuffle output goes to disk and can be re-fetched by an external service, so losing a worker costs a re-read, where Batcher pays a lineage recompute. Spark also decouples tasks from executors, running tens of thousands per stage, which buys load balancing, work stealing off a slow node and fine-grained recovery; Batcher’s unit is closer to one task per node, so a straggler’s whole partition is redone and skew cannot be diluted by over-partitioning. Structured Streaming is a real dataflow runtime with per-partition watermarks, a pluggable state backend and checkpointing that survives driver failure. And MLlib is a mature distributed *training* stack, which Batcher does not attempt.

The honest summary is narrower than a benchmark table suggests. On analytical queries over columnar data, an engine designed after Arrow existed has a structural advantage that has little to do with how well either system is engineered. On resilience at scale and streaming semantics, Spark is a decade of hard-won operational experience ahead, and closing that is a program of work rather than a design decision.

9 Kyber: an optimizer that learns

9.1 375 rules, and why that is safe

Seven phases run in a fixed order. The three rewrite phases iterate to a fixpoint because their rules are confluent; the cost-based and physical phases run once, because they make a decision rather than converge to one. Measured evidence enters at the last three: cardinality drives join ordering, refit cost coefficients drive fusion, and a bandit drives the physical choice.

Rules are grouped by family, boolean and Kleene algebra, sargability, temporal extraction, predicate inference, join shape,

set operations, window canonicalization, and expression families grouped by value domain, and discovered by a decorator registry. A rule set this size is safe only if each rule preserves results and no two interfere, and Kyber proves both mechanically. Each rule carries a plan-shape unit test and a differential test against DuckDB across nulls, empties and the type edges. The set as a whole is property-tested: a generator builds a random table and a random valid pipeline and asserts that the result under all 375 rules equals the result under none, order-independent. The same suite proves confluence and termination, the set converges to a deterministic fixpoint inside the production iteration budget, so the rules provably do not oscillate.

Two invariants bound what the algebraic families may do, and both have bitten. Integer arithmetic wraps on overflow, bit-for-bit with the JIT, so reassociation may only reduce $=$ and $\neq$, bijections of $\mathbb{Z}/2^{64}$, hence wrap-invariant, and never an ordered comparison, since wrapping breaks monotonicity. Three-valued logic means every boolean law is proven over $\{\top, \perp, N\}$; $x \wedge \neg x \rightarrow \perp$ fires only on never-null predicates. Some identities that look safe are refused, each refusal pinned by a test: $\text{pow}(x, 1)$ is not folded, because libm’s `pow` is not correctly rounded and the identity is therefore not provable, while $\text{pow}(x, 0)$ is folded because IEEE 754 specifies it as exactly one for every base. No timezone-conversion rewrite exists at all, a same-zone conversion is not the identity, because the engine nulls DST-ambiguous and nonexistent local times, and dropping the call would resurrect those rows’ values.

9.2 Provenance: rules gated on proof, not confidence

Every cardinality and column statistic carries *where it came from*, and the distinction between proven-exact and estimated-however-confidently is load-bearing rather than informational.

Two rule families fire only on EXACT provenance. One drops a `Distinct` over a column proven unique, skips a `Sort` over at most one row, prunes a constant sort key, and decides a column-to-column comparison from both columns’ proven

bounds. The other folds a subtree an exact cardinality proves empty. A sketch estimate may never drive either, because dropping a `Distinct` on a wrong guess is silent data corruption: the query returns quickly, with duplicate rows, and nothing turns red. Everything else in the optimizer may consume an estimate, because the worst outcome there is a slower plan.

Provenance also produced the sharpest defect this design has had, which is worth reporting because the fix generalizes. The gate deciding whether a query pays for stage-by-stage re-optimization read provenance as a proxy for “we do not know this size.” Those are different statements. Provenance describes where a number came from; it says nothing about whether the number was right. And the label is sticky, because the one-shot path never records an intermediate operator’s measured cardinality against its signature, so a shape can be estimated to within a percent of actual forever and still read as a guess. After the seeding fix above, TPC-H q5’s operands land within $1.0\times$ of actual and carried the default label anyway.

The gate now asks the evidence rather than the label. Core already records, per operator signature, the rows produced alongside the rows predicted, which the optimizer folds into a bounded window of $\log(\text{actual}/\text{estimated})$ to derive its correction factor. A shape is treated as confidently sized when that window holds at least a few observations and *every one of them* sits inside the re-optimization threshold. Reusing that threshold rather than inventing one is the point: it is the q-error at which the adaptive loop re-plans, so a history that never crossed it is a history in which a stage boundary would never have fired. The maximum is used rather than the mean, because alternating $4\times$ over- and under-estimates average to 1.0 and would read as flawless while being exactly the shape a stage boundary exists to correct.

Measured over the 22 TPC-H queries, counting routing decisions rather than time so a loaded machine cannot move the number (benchmarks/internals/adaptive_gate_routing.py): **15 of 22 queries routed to staging on a cold hub, and 0 once their estimates had demonstrably held up.** All 15 still route to staging when the recorded history shows the estimates missing, so the correction suppresses the cost without suppressing the case staging exists for. A cold hub knows nothing and behaves exactly as before, which keeps a first run’s plan unchanged.

9.3 Cardinality: four sources, strict precedence

Estimation starts from Selinger-style selectivities [28] and is superseded in fixed order by exact footer metadata, then by sketches built during execution (HyperLogLog [10] for distinct counts, KLL [15] for quantiles), then by measured cardinalities recorded under a plan signature. Every sketch is mergeable with a fixed seed, so one built per partition merges identically to one built on a single node: the algebra of Section 2 applied to statistics.

Getting this wrong is expensive in a specific way. TPC-H’s `orders.o_orderkey` carries no distinct count in any file footer, and without one the join estimator falls back to assuming a primary-key relationship, which is right for a fact-to-dimension join and catastrophic for a many-to-many key. At sf10 that priced one join at 1.5 million rows where the true output is about

six billion, and the executor set about materializing it, passing 110 GB resident. Kyber now measures the distinct count before the optimize that fixes join order for the whole query. Seeded, the planner picks the order DuckDB picks, and that query falls from $8.81\times$ to $3.95\times$ of DuckDB.

9.4 Cost, refit from runtime

Three axes, CPU, I/O, network, collapse to one scalar, with network shuffle bytes weighted at twice local bytes. Per-operator coefficients (inserting into a hash table costs more than probing it) are *recalibrated from measured operator times* once enough samples accumulate, clamped so timing noise cannot produce a degenerate model. Join ordering is exact dynamic programming over bushy plans at or below 12 tables, greedy to 25, unordered above that.

9.5 Two learned primitives

All of Kyber’s self-tuning reduces to two reusable pieces. Both read $O(1)$ sufficient statistics from the hub, keyed by plan signature and scoped to the host’s hardware profile, so a value learned on a 16-core laptop never steers a 96-core planner.

A **deterministic UCB1 bandit** [2] selects over a fixed arm set from measured per-arm latencies. There is no RNG and ties break by arm name, which matters more here than in a typical bandit application: a nondeterministic optimizer makes every performance regression unfalsifiable. Arm statistics are a *discounted* Welford state (n, μ, M_2) rather than raw sums, so variance is numerically stable and old observations decay as machine and data change. Its callers are the join-strategy choice, hash, broadcast, sort-merge, and the staged-versus-one-shot routing decision below.

An **OLS two-line crossover** learns a *threshold* where the bandit learns a choice. It fits $t \approx a + bx$ per algorithm and solves for the x (build bytes, or build rows) at which the cheaper-below algorithm is overtaken by the cheaper-above one. The result is clamped to a band around the shipped default, so one noisy early fit cannot send a threshold somewhere absurd, and an ambiguous fit returns nothing and the caller keeps the default. Its callers are the broadcast byte threshold and the sort-merge row crossover.

A third layer learns per-signature *scalars*: how big each join side was, how many rows a breaker shuffled, how far an aggregate collapsed its input, whether partial pre-aggregation paid off.

The invariance that licenses all of it. Every arm emits the same relation; a threshold selects between equivalent algorithms; a partition count only shards. Learning may change how much memory a query reserves, when it spills, how wide a morsel is, and which of several equivalent algorithms runs, and it may never change what the query returns. That is property-tested, tuned run equals untuned run, and a cold store reproduces untuned behavior byte for byte. Every read is best-effort: a malformed bucket, a degenerate fit, or a missing key yields the default, never an exception into planning.

9.6 When the loop measured nothing

TPC-H sf10 was losing to DuckDB by 5–12× on exactly the queries with the most joins, while single-table shapes sat at 1.2–1.7×. A split that clean by query *shape* is not a kernel problem. Measuring CPU time against wall time showed one query using 4.3 of 96 cores where DuckDB used 22.

All three causes were in the control plane and none produced a wrong answer. An adaptive stage handed the next stage an intermediate keyed by object identity, so statistics were filed under a name that died with the execution: sketches were recomputed every run, the learned store grew a dead entry per execution, and because a column absent from the store is by definition new, the learned generation advanced every run. The generation is part of the plan cache key, so the cache never hit. Zero hits, three misses, every run, forever. Second, the cache’s calibration epoch counted in a different clock from the refit it tracked and rolled over every second execution whether or not anything had been refit. Fixed, that query’s steady state fell from ~340 ms to ~165 ms. Third, the gate deciding whether to re-optimize between stages had been priced against an assumed 20–40 ms of control plane, when the loop actually materializes every join separately and forfeits both operator fusion and executor width. It is now a bandit over the two routes, rewarded with wall time.

The general lesson. All three defects were silent, each producing right answers while a load-bearing mechanism did nothing. A learning system needs tests that assert *the learning happened*, cache hit rates and generation stability and sketch reuse, not only that the answers are right. The answers are right either way.

10 Carbonite: resource decisions from measured footprints

Carbonite decides whether a plan is feasible, hands out memory reservations and shuffle credits, and decides when a query goes out of core. It never rewrites a plan and never computes a result.

10.1 Admission, and a memory model that is a ratio

Conventional admission control answers a boolean: does this plan fit? A rejection is useless to a planner, because it names no smaller thing to try. Carbonite returns the binding constraint *and* a counter-offer, a smaller credit window or a lower parallelism, that Kyber can re-plan around, and it names the *operator* that binds rather than the resource that ran out. This follows from the subsystem split: because Carbonite may not rewrite the plan, the only useful thing it can return is enough information for the layer that may.

Sizing used to come from the plan estimate alone, never from what operators actually used. Two choices in fixing that are forced rather than tuned.

Fit a per-input-row ratio, not an absolute peak. A family’s absolute peak depends on query size, so replaying a stored byte figure onto a differently-sized plan is simply wrong. A bytes-per-row figure is size-general: multiply by this plan’s estimated

input rows. It is the same reason the cardinality layer learns selectivity rather than a row count. One blended figure then feeds every memory decision, including batch sizing, which caps the row count by the widest learned per-row width so a workload of wide rows keeps its true byte working set inside budget where a fixed row count would not.

Fit the 80th percentile, not the median. Memory sizing is one-sided: over-provisioning costs a little headroom, under-provisioning costs the process. A median is by construction exceeded by half the runs, and the runs that exceed it are exactly the large, skewed, wide-row ones that fail. P80 rather than P95 because a family may have ten samples and the tail of a memory distribution is fat and noisy, so chasing a far tail fits noise. The result is clamped to a bounded factor of the plan estimate, and a cold store is a pass-through, so a first run is byte-for-byte the pre-learning behavior.

10.2 Credits, spill, and transport

Flow control is credit-based: one credit is one in-flight batch slot, and a producer blocks at zero, so a channel’s window is a direct bound on its memory. Three layers sit on that. Carbonite grants a static window and clamps any request to a ceiling. A TCP-like controller then grows and shrinks it from observed backpressure, with hysteresis so a channel does not oscillate. And a recurring shuffle warm-starts from the window past runs of that shape converged on. Learning moves only the starting point, so a stale value costs a few round trips of convergence and never correctness.

Two things about spill are not obvious. **Published shuffle output is memory nobody was reserving:** a mapper hands a bucket to the node’s transport layer and it stays resident until a reducer collects it, so with as many mappers as reducers a node holds its whole share of the shuffle against no reservation. It is now reserved, and it is deliberately the pool’s *first* spill victim, because a published bucket is finished work waiting to be collected. Spilling it stalls nobody, where spilling a half-built hash table interrupts an operator still using it. **And disk gets its own pressure ladder,** measured rather than accounted so a volume filled by a co-tenant is seen as readily as one this query filled. New buckets route to remote storage one level before the floor, because a bucket’s tier is fixed when it opens: by the time the floor is crossed, several are already streaming to a volume that cannot hold them.

Bulk data never enters the Ray object store; it moves over Arrow Flight [31]. Within a shuffle a reducer’s sources fall into three tiers chosen automatically: same-process reads straight from the local store, same-node-different-process memory-maps an Arrow IPC file and decodes zero-copy, and cross-node takes a credit-bounded stream. Measured on a cluster, an eight-producer gather runs at 33.6 GB/s over shared memory against 4.5 GB/s over loopback, and a miss falls back transparently, so single-node equals distributed regardless.

11 Execution, and one bound that was wrong

The executor pulls small batches through linear runs of operators and materializes only at pipeline breakers, in the style of morsel-driven parallelism [18]. Six operator-level results came out of that model, each verified against the sequential oracle before it was timed: a short-circuiting conjunctive filter (1.5–5.7 $\times$), dictionary-native comparison (19.6 $\times$), a runtime filter sunk across joins (1.9 $\times$), a deterministic parallel join map (1.8 $\times$), a bounded join fan-out (18 $\times$ less memory), and a real inequality-join operator (up to 900 $\times$). Individually they are unremarkable. Three points about them are not.

Two rewrites that are correct by construction. Most optimizations are justified by testing: apply the fast path, compare against the slow one, ship it when the tests pass. Two here are justified by an argument instead, which is what lets them run inside the correctness oracle rather than beside it.

The first short-circuits a conjunctive filter. Arrow has no selection vector, so where DuckDB narrows a predicate chain by masking, Batcher compacts: evaluate the cheapest predicate, gather the survivors, evaluate the rest against a shorter batch. That is only sound if skipping a row cannot hide an error it would have raised, so eligibility is decided by a syntactic test over the expression grammar. Predicates whose failures depend on a row’s *value*, arithmetic that can overflow, a strict cast, a string function, are excluded; predicates whose failures depend only on the *schema* are admitted. The second compares dictionary-encoded columns without decoding them, resting on the observation that an elementwise comparison commutes with a gather. Both produce bit-identical results for reasons that can be stated in a sentence, which is a stronger guarantee than a passing test suite.

One bound the system claimed and did not have. The streaming executor is supposed to hold the breakers’ state plus one batch per worker: a constant, independent of input size. Measured on a join-heavy shape it does, holding a few megabytes where the materializing path grows into hundreds.

The bound was false for joins, and the way it failed generalizes. The probe was written as though one input batch produces one output batch, which is true for every operator that preserves or reduces rows, and false for one that multiplies them. A skewed key turns each probe row into thousands of output rows, emitted as a single batch. Measured peak memory on such a query: 13.1 GB, with streaming on by default. Bounding the output, and then bounding the index arrays that describe it, brought that to 728 MB, an 18 $\times$ reduction, and cut the runtime by a factor of three, with identical results.

It survived a 46-test operator suite because every one of those tests is *correct*. A correctness suite cannot observe a memory property, and we had written the bound down in a comment rather than asserting it. It is now an operator-level assertion.

One place the algebra costs us. Inequality joins used to lower to a cartesian product with a filter above it, which is quadratic

in time and memory however few rows survive. They are now a real operator implementing IEJoin [16], which removed the wall: a case that took 3.4 seconds at ten thousand rows now takes under four milliseconds, and cases that did not run at all now do. It wins below roughly half a million rows and loses above a million, because DuckDB prunes blocks of the sorted input that cannot contain a match and we do not yet. That same pruning is what would make the operator distributable, so the mergeable-algebra constraint and the performance gap have one shared fix.

12 Evaluation

Setup. One Intel Xeon Platinum 8275CL at 3.00 GHz, 96 vCPU, 184 GiB, Linux 6.8; distributed results on an 8-node Ray cluster, 128 vCPU; GPU results on 8 $\times$ T4. DuckDB 1.5.5, Polars 1.36.1, Daft 0.7.21, Ray 2.56.0, PySpark 4.2.0, PyArrow 19.0.1. The harness refuses to report a timing for any query whose result does not match DuckDB, compared as an order-independent multiset. Timings are best-of- N warm.

Two hazards cost real time. A shared machine is not a benchmark machine: at load average 16–41 a repeated run swung $\pm 25\%$, so claims moving less than 2 $\times$ are quoted from a run at load under 5. And an in-place rebuild of the compiled extension silently replaces the engine mid-benchmark, so we stamp its timestamp around every run and discard any run in which it moved.

12.1 Does the algebra remove the fork?

The central claim is architectural, so its evidence is a testing argument and we state its strength precisely. Equation (1) is property-tested over random partitionings; an integration test asserts single-node equals multi-partition; and an operator matrix crosses {collect, spill, iterate, distributed} with {nulls, empty, one row, duplicates, $-0.0/\text{NaN}$, descending}, because the defects that reach production here are combinations of a non-default flag with a non-default path.

Two admissions. A test suite is not a proof, and we do not claim one. And the argument binds only operators that went through the admission criterion: we found one place it had not, where a watermarked windowed aggregate passed a distributability gate that never checked the watermark, so the same query returned different results single-node and distributed. The gate now refuses those plans loudly. The algebra removes the fork *within* the operator set; the residual exposure is at planner admission boundaries, which are ordinary code needing ordinary tests.

12.2 TPC-H

Table 3 is the headline, and it splits cleanly. Against Polars and Daft, Batcher leads by 1.8–2.2 $\times$ in geometric mean at both scales. Against DuckDB the result splits along the line Table 2 predicted: on identical Arrow input Batcher wins 21 of 22 at sf10 and is 1.89 $\times$ faster overall, and against DuckDB’s compressed native store it loses. q1 and q6 are essentially scans and sit at

Baseline	sf1		sf10		Note
	geomean b/x	wins	geomean b/x	wins	
DuckDB (native compressed store)	1.02×	12/22	1.49×	3/22	storage gap; Section 13
DuckDB (same Arrow input)	—	—	0.53×	21/22	like-for-like*
Polars [34]	0.55×	19/22	0.45×	17/22	
Daft [9]	0.48×	18/20	0.52×	13/20	wrong answers on q6, q15 [†]
Spark [1]	0.041×	22/22	0.074×	22/22	reads Parquet in timed region [‡]

Table 3: TPC-H, all 22 queries, correctness-gated, one 96-core node. b/x is $\text{Batcher}_{ms}/x_{ms}$: **below 1.00 means Batcher is faster**. Suite totals (ms), sf1 / sf10, Batcher 618 / 4,491; DuckDB 616 / 2,353; Polars 1,256 / 10,624; Daft 1,224 / 9,261; Spark 14,555 / 57,781. *Separate run: DuckDB executing the same zero-copy Arrow Batcher is given, rather than a compressed store it was allowed to build first; 4,453 vs 8,436 ms. [†]Daft covers the 20 queries it can express, raising on q21 and q22; Batcher agrees with DuckDB on 22 of 22 at both scales. [‡]Spark reads Parquet inside the timed region where the other four receive an in-memory Arrow handle, so its ratio is overstated by an unisolated scan cost. Ray Data is a streaming ML runtime without a relational optimizer, so a SQL suite is the wrong instrument for it; we evaluate it in Section 12.5, on the pipelines it is built for.

Operator case	DuckDB	Polars	PyArrow	Daft
groupby-sum	0.80	0.42	0.60	0.37
groupby-2key	0.62	0.40	0.54	0.22
global-sum	0.04	0.06	0.04	0.02
filter-count	0.07	0.03	0.00	0.03
join-agg	1.25	1.18	0.32	0.46
sort-limit	1.09	0.02	0.01	0.10
filter-project	0.79	1.10	0.06	0.49
window-rank	0.84	0.12	—	hangs
window-runsum	0.50	0.15	—	hangs
window-lag	0.71	0.04	—	hangs
window-sum-part	0.64	0.85	—	hangs
Batcher wins	8/11	9/11	7/7	7/7

Table 4: Operator mix at sf1, $\text{Batcher}_{ms}/x_{ms}$; below 1.00 is a Batcher win. All engines take the same in-memory Arrow input, so this carries none of Table 3’s scan asymmetry. PyArrow (Acero) has no window functions.

1.47×/1.52×, about what reading compressed pages instead of raw Arrow buys. But q21 (3.18×), q9 (3.79×), q7 (3.19×) and q5 (3.95×) are several times that, and those are engine work.

Correctness is not free for everyone. Daft gets TPC-H q6 wrong at both scales, dropping the `l_discount = 0.07` rows because `0.06 + 0.01` in IEEE double is `0.06999999999999999`; ground truth computed independently in PyArrow equals the official answer, and Polars makes the same mistake. This is why every timing is gated. A benchmark that does not check results reports the fastest wrong answer.

12.3 Operator mix and ClickBench

Table 4 is the cleanest comparison here: one input representation, one machine, no SQL front-end differences, no scan in the timed region for anybody. Batcher sweeps PyArrow’s Acero engine on all seven cases it can express and takes 8 of 11 from DuckDB and 9 of 11 from Polars.

On ClickBench [6] (43 queries, correctness-gated) Batcher wins 43 of 43 against DuckDB on identical Arrow input, 35 of 40 against Polars (the five losses are sub-3 ms queries where fixed overhead dominates), and 26 of 43 against its native store. Those losses are diagnostic: essentially all are high-cardinality

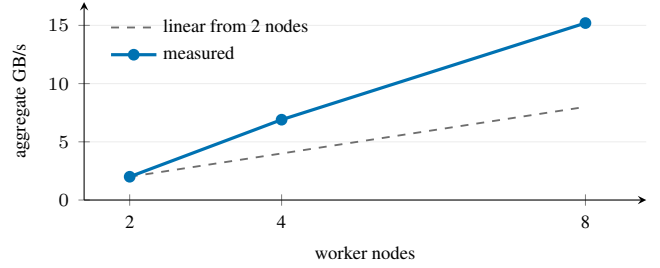


Figure 5: Aggregate all-to-all shuffle throughput against cluster width, with a linear reference anchored at two nodes. Four times the hardware returns 7.6× the throughput. The mechanism is the algebra: mappers publish pre-aggregated state rather than rows, so a hierarchical combiner tree plus credit-based flow control keeps per-node memory bounded as the cluster widens, and adding nodes adds reduce capacity instead of contention. A *single* reducer stays NIC-bound at ~2.7 GB/s, so the gain is in the aggregate, where every node reduces at once.

grouping and COUNT (DISTINCT) over string or near-unique keys, the shape behind the two remaining TPC-H losses too.

12.4 Scaling out

Figure 5 is the scaling result. It is superlinear, not merely linear, and the reason is worth separating from the headline: at two nodes the shuffle does not saturate the available reduce capacity, so part of the gain from 2 to 4 is recovered inefficiency. The claim we stand behind is the shape, that adding nodes adds throughput without a per-node memory penalty, not a promise of 1.9× per doubling forever.

The single-query picture matches. On TPC-H sf10 q6 with every engine reading the same S3 Parquet and correctness-gated, Batcher over 64 partitions runs 224.5 ms against Daft on its Ray runner at 535.8 ms and DuckDB-on-Arrow single-node at 457.3 ms, and Daft returns the wrong answer described above. Across a pipeline suite at sf1/sf10/sf100 Batcher is ahead of Ray Data on every pipeline and leads Daft on joins (1.7–2.2×), group-by and metadata-only counts.

Where linear scaling stops, measured. Two ceilings bound the claim and both are ours. *Shuffle cost is quadratic in partition count*: on a forced distributed 6M-row grouped sum, 16

partitions cost 1,031 ms and 128 cost 93,189 ms, close to the $64 \times$ a P^2 pair count predicts, so about 5.7 ms of fixed cost per partition *pair*. One partition per cluster CPU is the obvious sizing and is exactly the setting that falls off the cliff. And *distributed scan throughput does not scale with the data*: on the object-store read path, aggregate throughput across 8 workers measures roughly 90 MB/s and is approximately constant from sf10 to sf100, where a competitor driving more parallel coalesced range reads per node is several times faster. So the scaling claim here is about the *shuffle*, the part the algebra governs. The read path in front of it is separate work, it is not good yet, and we would rather say so than quote the shuffle number as though it characterized an end-to-end petabyte run.

12.5 GPU and ML workloads

These are the workloads Ray Data was built for, which makes it the right yardstick and the wrong thing to keep score against. The question worth answering is whether an engine that treats decode and inference as relational operators can hold its own against a runtime purpose-built for them. It can. On $8 \times T4$ with real models: ResNet-50 batch inference $2.05 \times$, image embeddings $1.98 \times$, fractional-GPU packing $1.96 \times$, video-clip inference $3.6 \times$, training-data ingest $3.0 \times$, multimodal JPEG-to-GPU $1.3\text{--}2 \times$.

Three figures are much larger, and should not be quoted as general execution speed: audio feature extraction $12.5 \times$, LLM batch inference $11.1 \times$, text embeddings $47 \times$. They come from two scheduling mechanisms rather than kernels. Pools that load a model once per session and reuse it across executions, so the ratio scales with how much of a job is model load and shrinks as jobs grow. And native in-engine preprocessing, where mel-spectrogram and MFCC run as Arrow kernels instead of a per-batch Python UDF. Neither is an architectural advantage; both are choices any runtime could adopt, and we expect the gap to close. The durable claim is the $2\text{--}3.5 \times$ range, and what it shows is that putting AI operators inside a query engine costs nothing against a specialized runtime.

Two honest exceptions. On a single maximally-large compute-bound job both engines saturate the same GPU at the same FLOPs and the ratio is near parity. And our own image ingest began this work *losing* to Ray Data and Daft at ~ 350 img/s, because our decode kernels ran serially and the executor sized its pool from the batch count; five fixes took it to $\sim 5,700$ img/s.

A note on the harness. An earlier version of our adapter registered each table as a single block, which is Ray Data’s unit of parallelism, so a 6M-row table ran on one core of a 96-core machine. That was our defect, not a property of Ray Data. Every number here is from the corrected adapter, which writes tables as untimed setup and reads them back through the Parquet connector with row groups sized to Ray’s own target block size.

13 What is not done yet

A roadmap beats a scoreboard, so this section is specific about what does not work yet. Each item is measured, and together

they are four programs of work rather than a list of bugs: a storage engine, which decides the sf10 result and is the most separable because it sits below the algebra; radix-partitioned high-cardinality aggregation, the one systematic kernel gap; durability at scale, whose parts are independent and so the most incremental; and a real streaming runtime, which Section 3 argues is bounded work on the runtime rather than a second engine.

The adaptive loop’s cost is half-fixed, and we have not re-timed it. At sf10 it used to lose to one-shot planning on 20 of 22 queries, 3,889 ms against 4,669 ms, for two reasons. One is now closed: the gate read a provenance label instead of the measured error history, and firing on 15 of 22 queries that did not need it (Section 9.2). The other is not: the loop still materializes one breaker per stage, so when it does fire it forfeits operator fusion and the streaming executor’s width. We report the routing count rather than a new suite time because the machine these measurements ran on was shared, and at load average 16–41 the harness swings $\pm 25\%$, which is larger than the effect. The re-measurement on a quiet box is the open item.

We are not finer-grained than Spark AQE at the stage clock. An earlier version of our own documentation claimed otherwise. It is stage-boundary adaptation with the same granularity, off below 20M input rows. What is defensible is the pair of clocks around it, not that one.

Storage is the comparator we do not sweep. Three of 22 TPC-H queries at sf10, 26 of 43 ClickBench, 6 of 11 operators, all against DuckDB’s native store, and on the same Arrow input Batcher wins essentially all of them. The residue is compression, zone-map skipping and late materialization: a storage-engine program, and it should be costed as one.

High-cardinality grouping is the one systematic kernel gap. It accounts for essentially every ClickBench loss and the two remaining TPC-H losses to DuckDB. When the group count approaches the row count, per-thread partials stop collapsing. One named work item.

And the operational gaps of Section 8: a memory-resident shuffle with lineage recompute rather than re-fetch, task granularity near one per node, a single global watermark with in-memory streaming state, and no compressed native storage format or string view. Each is a known program of work rather than a surprise.

14 Related work

Batcher’s execution is morsel-driven [18] over a vectorized Arrow representation in the MonetDB/X100 lineage [5], with a compiled path for scalar expressions restricted to a subset carrying a proven-identical fallback. Volcano [11] established the operator/exchange separation; the departure is that Batcher has no distributed operator to separate.

Mid-query re-optimization [14], progressive optimization [23] and eddies [3] are the lineage of the stage-boundary clock, surveyed in [8], with Spark AQE [29] its closest deployed relative. Learned optimizers [21, 22] attack estimation with models; Batcher takes the lighter route of sketches [10, 15], refit cost coefficients and a bandit [2] over result-equivalent strate-

gies, which buys reproducibility and a cold start that is exactly the untuned system. [19] is the standing argument for why measured cardinalities beat better estimators, and Structured Streaming’s framing of a stream as an unbounded table [36] is the one Section 3 adopts.

15 Conclusion

The architectures most teams run were designed for a machine that was too small, a schema that never changed, a world where machine learning happened somewhere else, and a corpus that was mostly rows and columns. All four assumptions have expired. The workarounds they force have become so familiar that they read as the cost of doing business: a cluster kept warm so small queries are bearable, a shuffle partition count tuned by hand, a second copy of the data for the ML team.

Batcher is an argument that they are not. One mergeable triple per operator removes the split between the fast engine and the scalable one, so the same code runs on a laptop and on a cluster: speed is not re-earned at scale, and scale is not paid for in advance. The same constraint makes measurement transferable, which is what lets the engine plan from evidence instead of from constants chosen years ago for hardware nobody is running. It makes batch the bounded case of streaming rather than a second execution model. And because decode, embedding and inference are expressions in that algebra, the pipeline that turns raw data into model input is one optimized plan instead of three systems and two copies.

What the design does not buy is storage, and that gap is real: Batcher matches DuckDB at sf1 and loses at sf10 on compression, zone maps and late materialization. Nor does it buy operational maturity. Spark’s resilience and streaming guarantees are a decade of production experience ahead, and Section 8 says so plainly, because a paper that hid it would not be worth reading. Both are localized to specific work rather than to the idea, which is the most useful thing we can report.

Reproducibility. Every number comes from a committed harness: `benchmarks/run.py` drives the engine adapters and `harness.py` enforces the correctness gate, refusing to time any query whose result does not match DuckDB. The harness records the engine build profile with each run, so a result cannot silently be attributed to a different build. Open items before archival publication: Table 3 reports a single run per scale and needs repetitions with confidence intervals, and no ablation yet attributes the end-to-end total to the individual mechanisms of Section 11.

References

- [1] Michael Armbrust, Reynold S. Xin, Cheng Lian, Yin Huai, Davies Liu, Joseph K. Bradley, Xiangrui Meng, Tomer Kaftan, Michael J. Franklin, Ali Ghodsi, and Matei Zaharia. Spark SQL: Relational data processing in Spark. In *ACM SIGMOD International Conference on Management of Data*, 2015.
- [2] Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2–3):235–256, 2002.
- [3] Ron Avnur and Joseph M. Hellerstein. Eddies: Continuously adaptive query processing. In *ACM SIGMOD International Conference on Management of Data*, 2000.
- [4] John Black and Phillip Rogaway. Ciphers with arbitrary finite domains. In *Topics in Cryptology — CT-RSA*, 2002.
- [5] Peter A. Boncz, Marcin Zukowski, and Niels Nes. MonetDB/X100: Hyper-pipelining query execution. In *Conference on Innovative Data Systems Research (CIDR)*, 2005.
- [6] ClickHouse, Inc. ClickBench: A benchmark for analytical databases, 2024.
- [7] Jeffrey Dean and Sanjay Ghemawat. MapReduce: Simplified data processing on large clusters. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2004.
- [8] Amol Deshpande, Zachary Ives, and Vijayshankar Raman. Adaptive query processing. *Foundations and Trends in Databases*, 1(1):1–140, 2007.
- [9] Eventual Computing. Daft: A distributed query engine for large-scale multimodal data, 2024.
- [10] Philippe Flajolet, Éric Fusy, Olivier Gandouet, and Frédéric Meunier. HyperLogLog: The analysis of a near-optimal cardinality estimation algorithm. In *Conference on Analysis of Algorithms (AofA)*, 2007.
- [11] Goetz Graefe. Volcano—an extensible and parallel query evaluation system. *IEEE Transactions on Knowledge and Data Engineering*, 6(1):120–135, 1994.
- [12] Jim Gray, Surajit Chaudhuri, Adam Bosworth, Andrew Layman, Don Reichart, Murali Venkatrao, Frank Pellow, and Hamid Pirahesh. Data cube: A relational aggregation operator generalizing group-by, cross-tab, and sub-totals. *Data Mining and Knowledge Discovery*, 1(1):29–53, 1997.
- [13] International Data Corporation. Worldwide global DataSphere structured and unstructured data forecast, 2025–2029. Technical Report US52800025, International Data Corporation, 2025.
- [14] Navin Kabra and David J. DeWitt. Efficient mid-query re-optimization of sub-optimal query execution plans. In *ACM SIGMOD International Conference on Management of Data*, 1998.
- [15] Zohar Karnin, Kevin Lang, and Edo Liberty. Optimal quantile approximation in streams. In *IEEE Symposium on Foundations of Computer Science (FOCS)*, 2016.
- [16] Zuhair Khayyat, William Lucia, Meghna Singh, Mourad Ouzzani, Paolo Papotti, Jorge-Arnulfo Quiáné-Ruiz, Nan Tang, and Panos Kalnis. Lightning fast and space efficient inequality joins. *Proceedings of the VLDB Endowment*, 8(13), 2015.
- [17] Andreas Kipf, Thomas Kipf, Bernhard Radke, Viktor Leis, Peter Boncz, and Alfons Kemper. Learned cardinalities: Estimating correlated joins with deep learning. In *Conference on Innovative Data Systems Research (CIDR)*, 2019.
- [18] Viktor Leis, Peter Boncz, Alfons Kemper, and Thomas Neumann. Morsel-driven parallelism: A NUMA-aware query evaluation framework for the many-core age. In *ACM SIGMOD International Conference on Management of Data*, 2014.
- [19] Viktor Leis, Andrey Gubichev, Atanas Mirchev, Peter Boncz, Alfons Kemper, and Thomas Neumann. How good are query optimizers, really? *Proceedings of the VLDB Endowment*, 9(3):204–215, 2015.
- [20] Michael Luby and Charles Rackoff. How to construct pseudorandom permutations from pseudorandom functions. *SIAM Journal on Computing*, 17(2):373–386, 1988.
- [21] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Nesime Tatbul, Mohammad Alizadeh, and Tim Kraska. Bao: Making learned query optimization practical. In *ACM SIGMOD International Conference on Management of Data*, 2021.
- [22] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Chi Zhang, Mohammad Alizadeh, Tim Kraska, Olga Papaemmanouil, and Nesime Tatbul. Neo: A learned query optimizer. *Proceedings of the VLDB Endowment*, 12(11), 2019.
- [23] Volker Markl, Vijayshankar Raman, David Simmen, Guy Lohman, Hamid Pirahesh, and Miso Cilimdzic. Robust query processing through progressive optimization. In *ACM SIGMOD International Conference on Management of Data*, 2004.
- [24] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. Ray: A distributed framework for emerging AI applications. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2018.
- [25] MosaicML / Databricks. StreamingDataset: Fast, accurate streaming of training data from cloud storage, 2023.
- [26] Mark Raasveldt and Hannes Mühleisen. DuckDB: An embeddable analytical database. In *ACM SIGMOD International Conference on Management of Data*, 2019. Demonstration.
- [27] David Reinsel, John Gantz, and John Rydning. The digitization of the world: From edge to core. IDC White Paper US44413318, International Data Corporation, November 2018.
- [28] P. Griffiths Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price. Access path selection in a relational database management system. In *ACM SIGMOD International Conference on Management of Data*, 1979.
- [29] The Apache Software Foundation. Adaptive query execution — Spark SQL performance tuning guide, 2024.
- [30] The Apache Software Foundation. Apache Arrow: A cross-language development platform for in-memory analytics, 2024.
- [31] The Apache Software Foundation. Arrow Flight RPC, 2024.
- [32] The Ray Team. Ray data: Scalable datasets for ML, 2024.
- [33] Transaction Processing Performance Council. TPC Benchmark H standard specification, 2022.
- [34] Ritchie Vink and Polars contributors. Polars: DataFrames for the new era, 2024.
- [35] Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma, Murphy McCauley, Michael J. Franklin, Scott Shenker, and Ion Stoica. Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2012.
- [36] Matei Zaharia, Tathagata Das, Haoyuan Li, Timothy Hunter, Scott Shenker, and Ion Stoica. Discretized streams: Fault-tolerant streaming computation at scale. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2013.